Hajee Mohammad Danesh Science and Technology University
MSc in CSE
Dept. of Computer Science
Course: Software Project Management
Course Code: CSE 531
Prepared by: Professor Dr. Ashis Kumar Mandal

Reference Books:
1. Introduction to Software Project Management by Adolfo Villafiorita
2. Software Engineering by Ian Sommerville

Making IT Right: Managing Goals, Time, and Costs

Project Value: Aspects to Consider

Three main factors determine the value generated by a project:

- **Direct and indirect value.** The value of a project does not refer necessarily and only to the revenues it generates directly and through its outputs. Considerations relative to the social and environmental impact, image and publicity, entering a new market, and know-how acquired are some of the considerations that could add or subtract value from a project.
- **Sustainability.** Many IT projects start without an idea or a strategy to sustain their outputs. Thus, the outputs of a project might not live long after a project and its resources end. Taking into account the operational costs of a project's outputs and the way in which the project outputs will survive after a project ends is an important consideration to understand whether a project is worth doing.

  Alignment with the strategic objectives of the organization. Ensuring that the project aligns with the goals of an organization is an essential point to consider before a project is worth starting. Alignment with the strategic objectives can determine the priority of a project.

Project Risks: Aspects to Consider

Various factors determine the risk profile of a project. Among them are

- **Resource availability.** Projects require the availability of resources—human, financial, and technical—in specific time frames. Although it might be difficult to preempt the required resources in advance, a check on the project's needs is a good sanity check to verify whether a project is worth pursuing.
- **Timing.** Many projects have specific time windows for the delivery of their outputs. Deliver too early or too late and the outputs of the project might be useless. Consider, for instance, a project to build a rocket to reach another planet. The actual launch can occur only on a specific time frame, to take advantage of the relative position of planets.

Deliver the rocket too early, and docking and maintenance might become an issue. Deliver too late and you might lose the opportunity to launch.

- **Technical difficulty or uncertainty.** The success of many projects relies on the actual capability of solving various technical challenges. Pointing out what these challenges are, understanding the level of risk associated with such challenges, and possible corrective or alternative courses of action are important in determining the values and risks of a project.
- **Project environment and constraints.** Projects are influenced by various constraints, both internal and external. Various internal and external stakeholders will have an interest in positively or negatively influencing a project. Regulations and standards can severely limit what can be done on a project.

Techniques to Assess Value and Risks

Financial Methods

- **Payback:** which measures how long it will take to return a project's investment. The payback is the year at which the project covers all expenses and starts earning. The shorter the payback, the better.
    - One of the issues with payback is that it does not take into account total profit.
    - This second issue is that it does not measure the efficiency with which the money invested in the project is paid back.
- **Return of Investment** To overcome some of the limitations of the payback method, another technique that is often employed is the return of investment (ROI), which measures how much we get back for each dollar invested. ROI is calculated from the annual profit, defined as the average profit per year and computed by dividing the profit by the duration of a project:

$$\text{Annual profit} = \frac{\text{incomes} - \text{expenses}}{\text{project duration}}$$

- The ROI is then computed by dividing the annual profit by the total project Expenses:

$$\text{ROI} = \frac{\text{annual profit}}{\text{expenses}}$$

- **Net Present Value** Payback and ROI do not take into account the effects of inflation, namely, the fact that an amount of money in the future has a lower value than the same amount available now. The net present value technique (or NPR for short overcomes this issue by taking into account the inflation rate. In particular, when using NPR, profit and losses are computed using the following formula:

$$\text{Value} = \frac{1}{(1+r)^i} * \text{amount}$$

where r is the inflation rate, amount is the net profit at year i, and value is the current value of amount. Note that the first project year is year 0.

-

Applying Financial Methods: An Example

Consider two projects, called “Project A” and “Project B,” for which we have estimated expenses and incomes as described in Table. In particular, the table shows that project A is not profitable in the first 2 years and then starts earning. Project B has a similar behavior, but both expenses and incomes are higher.

Suppose we have the resources to start only one of the two projects and we use a financial method to choose which project to activate.
If we use the payback method, we select Project A, since it has a shorter payback period. In fact, year 0 is forecast to end with losses, for Project A. So will year 1. At the end of year 2, however, the financial statement will show earnings of e10, 000.

Assessing Two Projects Using Financial Methods

	Project A	*Project B*
Year 0	−€20,000.00	−€30,000.00
Year 1	−€10,000.00	−€30,000.00
Year 2	€40,000.00	€50,000.00
Year 3		€100,000.00

Project B’s payback is 4 years. Years 0 and 1 end with losses. In year 2, project B earns profits of e50,000, but these are not yet sufficient to cover the expenses of the first 2 years. In year 3, however, the project pays back the initial investment.

If we use the ROI method, we select Project B, since it has the highest ROI. Project A, in fact, has an ROI of 11%, computed as follows:

$$\text{Annual profit} = \frac{€40{,}000 - (€20{,}000 + €10{,}000)}{3} = €3333$$

$$\text{ROI} = \frac{€3333}{€300{,}00} = 11\%$$

The ROI of Project B, whose calculation we leave to the reader, is 38%.

Score Matrices

Financial methods help determine the financial viability of a project, but tell nothing about the project characteristics that are not measurable with profits and losses.

A score matrix is a list of project criteria, each of which is assigned a weight, which measures the importance the criteria have for us or for the organization we work for. The criteria highlight the desirable and undesirable aspects of a project; the weights of desirable features are positive numbers (e.g., from 1 to 5) and the weights of undesirable features are negative numbers (e.g., from −1 to −5).

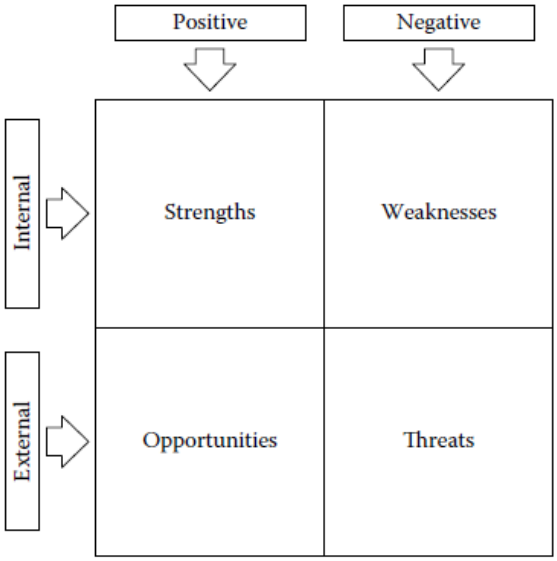
When we evaluate a project using a score matrix, we measure how well the project satisfies each criterion we have identified, for instance, by assigning a number from 1 (very low) to 5 (very high). **We then multiply the scores with the weights and sum all values. Projects scoring a higher value are more desirable than projects with a lower score.**

A Score Matrix Example: **According to the data, "Project 1," the one with the highest score, is preferred over the others.**

			Project 1		Project 2		Project 3	
Factor	*Description*	*Weight*	*Value*	*Total*	*Value*	*Total*	*Value*	*Total*
Profit >30%	The project will yield a profit >30% if no exceptional events occur	3	4	12	4	12	4	12
Low-risk profile	The project does not present particular risks. That is, there is no risk with a very high impact	2	2	4	3	6	3	6
Schedule is not tight	Project delivery does not require activities to be performed in a very tight schedule	3	3	9	2	6	2	6
Manageable complexity	The complexity is manageable	2	1	2	1	2	1	2
Consistent with current business	The project is mainstream with the activities of the organization	1	1	1	1	1	1	1
Stakeholders	Stakeholders are difficult to manage	−4	2	−8	4	−16	3	−12

SWOT Analysis (The strengths, weaknesses, opportunities, and threats analysis) The SWOT analysis is usually performed on a two-by-two matrix, The analysis proceeds by identifying the strengths, weaknesses, opportunities, and threats related to the project under analysis, and by listing them in the matrix shown in Figure

SWOT Analysis for a company:



SWOT Analysis for the enrollment software

Strengths - User-friendly interface - Automation of tasks - Strong data security - Scalability - Customizable features	Weaknesses - High initial cost - Complex setup - Limited system integration - Dependence on internet connectivity
Opportunities - Growing demand for online education - Partnerships with institutions - Market expansion into different educational sectors - Technological advancements (AI, analytics)	Threats - Strong competition - Data privacy regulations - Technology obsolescence - Cybersecurity risks

The Project Feasibility Document

The project feasibility document closes the assessment phase by describing the main characteristics of a project and by analyzing, using the techniques presented above, the value and risks of a project. The document can be used to formally authorize the start of a project.

Thus, a feasibility document contains at least the following information:

- **A statement of work,** which describes what the project will accomplish

- **The business objectives** of the project and its outputs (value) and information about the business model, if relevant
- A summary of the project budget, which forecasts expenses and incomes
- A summary of the project milestones, which is a rough schedule of the project identifying the most important events
- An analysis of the stakeholders
- The project risks
- Possible alternatives to the project, such as a make-or-buy decision
- An evaluation of the project

Formalizing the Project Goals

Defining project scope is one of the most delicate activities in setting up a project
The project scope is fixed in a project scope document. In particular, it is a good idea to include at least the following information in a project scope document:

- **Project goals and requirements,** which describe what we intend to achieve with the project and the main characteristics of the project and its outputs
- **Assumptions and constraints,** which describe the conditions which have to be met for the project to succeed
- **Project outputs and control points,** which describe the outputs of the project and, in some cases, a rough timing of their delivery
- **Project Roster**, which describes the who.

Project Goals and Requirements

The project goals and requirements define what is inside and what is outside the scope of a project and the characteristics of the project outputs.
Two good practices help make the goals **SMART** and assign them priorities using the **MoSCoW classification.**

- SMART stands for simple, measurable, agreed upon, realistic, and time bound. A goal is SMART when it has the qualities specified by the acrynom. Thus, a SMART goal is simple in its formulation, so that there are no ambiguities in its interpretation; it has measurable criteria to understand whether it has been achieved or not; it is agreed to by all the stakeholders; it can be reached with the resources available in the project; and finally, it has a date by which it has to be reached.

- MoSCoW allows the project manager to assign a priority to the goals. The acronym stands for must have (features that are essential), should have (features that are important but not essential), could have (features that would be nice to have), and won't have (features that will not be included in the product).

Project Assumptions and Constraints

Project Assumptions Assumptions are factors considered to be true for planning purposes. They help identify potential risks.

Examples:

- **Resource Availability**: Necessary personnel and equipment will be accessible.
- **Stakeholder Support**: Stakeholders will provide required input and approvals.
- **Market Stability**: Market conditions will remain consistent throughout the project.
- **Technology Reliability**: Chosen technology will function as expected.
- **Regulatory Environment**: No significant changes in laws affecting the project.

Project Constraints Constraints are limitations that impact the project's scope, schedule, budget, and quality.

Examples:

- **Time**: Project must be completed by a specific deadline.
- **Budget**: Fixed budget that cannot be exceeded.
- **Scope**: Limited to specific features or functions.
- **Quality**: Must meet established quality standards.
- **Resources**: Limited availability of skilled personnel or equipment.

Project Outputs and Control Points
Project Outputs are the results or deliverables produced by a project, while **Control Points** are critical checkpoints used to monitor progress and make decisions throughout the project lifecycle. Together, they help ensure that projects are completed successfully, on time, and within budget, while meeting the needs of stakeholders.
A deliverable is defined in Project Management Institute (2004) as a unique, measurable, and verifiable work product.
Milestones are defined by the Project Management Institute (2004) as a significant event in the project. Milestones are identified, at a minimum, by a label and a date, and they are typically used to highlight, significant control points, in the plan.

Milestones can be used in different ways:

- **To form (and present) a high-level roadmap of the project:** milestones represent how the project unfolds in a series of significant events.
- **As a verification point in the project:** milestones identify control points in the project, which serve as a general "orientation" mechanism to steer the project in one direction or another.
- **As a gate in the project:** milestones clearly separate different phases of the project; if the goals of the milestone are not achieved, the transition to the next phase is blocked.

EXAMPLE

Table below shows the standards the European Union enforces for the specification of deliverables produced by the research projects it sponsors. In particular, the following information is associated with each deliverable:

- A unique identifier, typically an integer number
- The name of the deliverable
- The nature of the deliverable, which can be one of the following: a report (R), a prototype (P), demonstrator (D), or other (O)
- The dissemination level, which, simplifying a bit on the EU rules, can be public (PU), restricted to the project team (RE), or restricted to the project stakeholders (CO)
- The delivery date, expressed in the number of months after the start of the project
- The partner (team member) responsible for the delivery of the project.

Del. ID	*Deliverable Name*	*Nature*	*Dissemination Level*	*Delivery Date (Project Month)*	*Responsible Parter*
1	Requirements	R	CO	0	FBK
2	Architecture UML diagram	R	CO	1	FBK
3	Software	P	PU	6	FBK
4	User manual	R	PU	12	FBK

Table: An Example of Deliverables

Similarly, Table below shows (a subset of) the milestones of a European Research project. Milestones have the following information:

Milestone Number	*Milestone Name*	*Date (Month from Start)*	*Means of Verification*
M0.1	Kick-off	1	Kick-off meeting done, meeting minutes available, project collaboration tools available.
M0.2	Experimental sites 1&2 up and running	12	Experimental sites 1 & 2 up and running.
M0.3	Midterm review	18	At least 80% of activities starting before M18 have started; at least 80% of activities ending before M18 have ended.
M0.4	Experimental sites 2&4 up and running	24	Experimental sites 3 & 4 up and running.

Table An Example of Milestones

Project Roster

The project roster is the list of people participating in the project, together with their role and other information, such as the contact point. The project roster is a simple practice that allows the project manager to identify the project stakeholders and, in the process, simplify the definition of a project communication plan and favor team interaction.

Deciding the Work

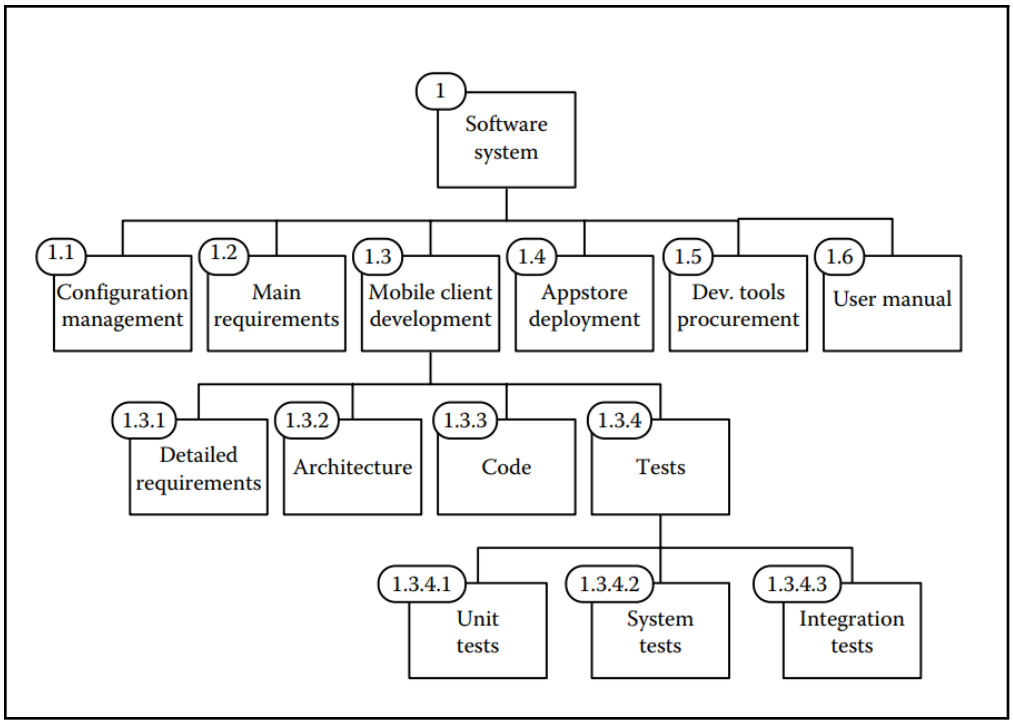
A work breakdown structure (WBS) is a project management system that breaks projects into smaller, more manageable components or tasks. It is a visual tool that breaks down the entire project to make it easier to plan, organize, and track progress.

Example: Figure shows a graphical representation of a WBS for the development of a software application for mobile phones.

The WBS is structured in four levels.

The first level of decomposition contains the main activities, including the procurement of tools we need for development (1.5) and writing the user manual (1.6). Tests (1.3.4) are organized in three different types of activities: unit tests (1.3.4.1), system tests (1.3.4.2), and integration tests (1.3.4.3).

WBS example



WBS Decomposition Styles

- **In a product-oriented WBS,** the decomposition proceeds by identifying the items that must be developed to build deliverables.
- **In a process-oriented WBS,** the decomposition proceeds by taking into account the activities that are necessary to carry out the project.
- A hybrid WBS contains both process- and product-oriented nodes

WBS Construction Methodologies

- **In the top-down approach,** the construction of the WBS proceeds from the top level down to the leaves. It is usually best suited for projects that are well known or whose structure is clear.

- **In the bottom-up approach,** the process proceeds in the opposite way. First, the leaves of the WBS are identified and then these are grouped in homogeneous items, thus giving structure to the WBS. It is best suited for projects that are new for a company or for the team responsible for their development and they work better with brainstorming sessions in which the whole team is involved.

Estimating

Effort, Duration, and Resources

Estimation is the process that determines the requirements to carry out an activity. These are expressed in terms of

- **Duration**, namely, how long an activity will last It is measured in calendar units, such as days, weeks, months, and years
- **Effort,** namely, the amount of work necessary to complete an activity and is measured using man-hours, man-days, man-months, or man-years meaning, respectively, the amount of work expressed by one worker in an hour, a day, a month, or a year. Thus, for

instance, an activity requiring 40 man-hours can be completed by a person working for 40 h.

- **Resources**, necessary to complete an activity. The main resource needed for software development projects is manpower, which is expressed in terms of units of work, that is, the effort that can be produced per calendar period

For many tasks, a simple relationship links effort, duration, and manpower:
D = E/ M
where D is the duration of an activity, E is the effort required by the activity, and M is the manpower required to carry out an activity. Of the three variables, one is usually estimated, another chosen, and the third computed.
Finally, manpower is determined by the work calendar and the percentage of availability

Finally, note that some activities might also require one to specify other types of resources, namely, materials and equipment:
Material is necessary for certain activities and is consumed while work progresses.
Equipment includes the tools required for carrying out work.
Materials are typically fed into **equipment** to produce the final product or facilitate the development process.

In software development, materials could include things like:

- External datasets for machine learning model training.
- Consumable resources such as printouts for planning or design diagrams.
- External API usage or data storage, depending on whether they are consumed/paid per use.

In software development, equipment could include:

- **Computers**: For coding, designing, and running simulations or compiling code.
- **Servers or Cloud Infrastructure**: Virtual machines used for hosting databases or applications.
- **Mobile Testing Devices**: Phones, tablets, and laptops for testing applications across platforms.

The "Quick" Approach to Estimation

- The simplest and probably most commonly used approaches to estimation are expert judgment and analogy. The project manager, possibly in collaboration with the team or other experts, provides estimations using his/her experience or by looking at similar projects. The approach has the advantage of being very fast and simple.

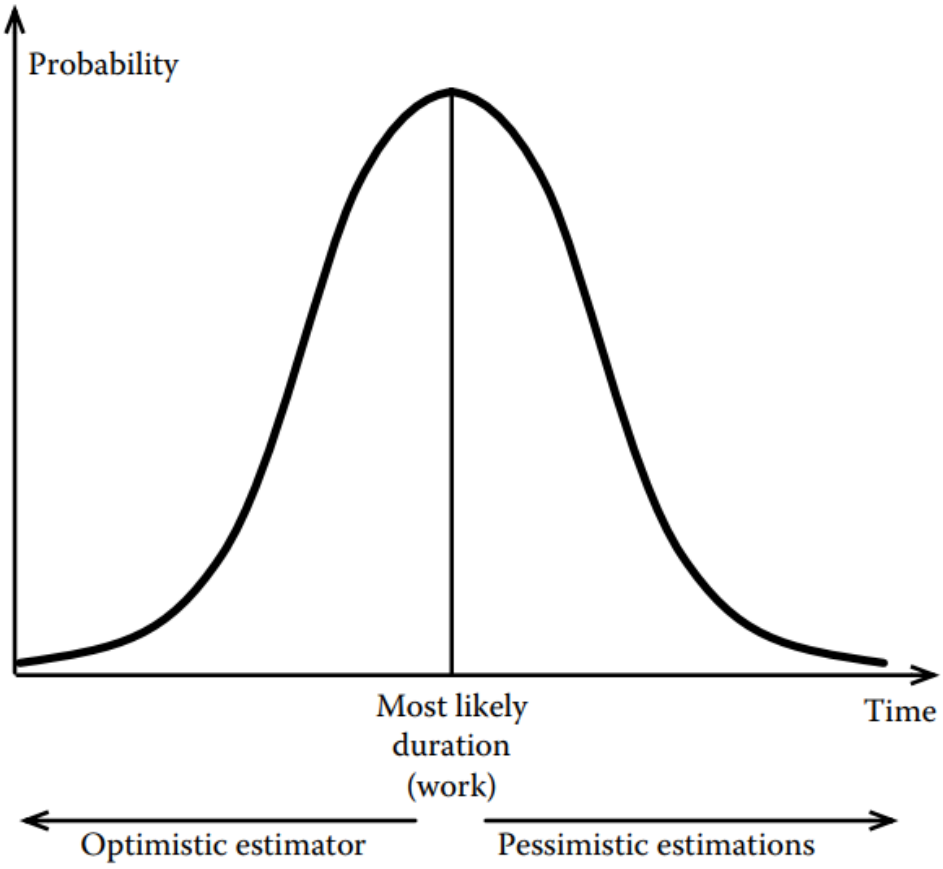
- These estimations can either proceed bottom-up or top-down. In the bottom-up approach, the manager provides an estimation for each leaf of the WBS. If the estimations are effort-based, these can be easily propagated upward, thus determining the effort required for each node of the WBS.

- In the top-down approach, the process is reversed. The manager provides an estimation for the overall project. If the estimations are effort-driven, the effort of each activity of the WBS is then determined by distributing it to the lower levels.

(Software) project managers make many implicit and explicit assumptions when estimating the resources necessary for an activity and many estimations are based on a number of “ifs.”

For instance, consider the problem of estimating how long it will take to complete a “requirement definition” activity, which produces a software requirements document. The reasoning could proceed as follows: if the final requirement document will be about 100 pages and if analysts can produce about 2.5 pages per hour, then the work will require about 40 man-hours. If one person will be able to dedicate about 80% of her time, then the actual duration will be about 50 h. The final estimation depends upon all these assumptions. For instance, if the estimation on the number of pages to write is increased by 10% and the productivity is reduced to 2 pages per hour, the duration of the tasks increases to 68.75 h.

We can look at estimations as random variables characterized by a mean and a variance. when we provide an estimation, we give our best guess of a task duration. An important implication, however, comes from considering the error between our estimation and the mean value of the actual duration (work) of a task. In fact, estimations (and estimators) can be optimistic if they are below the mean value or pessimistic, if they are above the mean value.



Estimations as random variables

Three-Point Estimation (PERT)

- **Description**: Utilizes three estimates for each task: optimistic (O), pessimistic (P), and most likely (M). The expected duration is calculated using the formula:

Provides a more nuanced view of uncertainty; Can be time-consuming; depends on the ability to accurately estimate the three points.

$$\text{Expected} = \frac{O+4M+P}{6}.$$

Algorithmic Techniques

Algorithmic techniques determine the effort or the duration required for developing a software system, given some of its characteristics. In other words, given a set of measurable features of a system $x_1, \ldots, x_n$, an algorithmic technique defines a function $f(x_1, \ldots, x_n)$, such that $f(x_1, \ldots, x_n)$ returns the effort, duration, and team size required to develop a system described by $\langle x_1, \ldots, x_n \rangle$.

Algorithmic techniques offer reliable, repeatable, and objective estimates for key software development parameters, such as delivery dates and project budgets. However, they have limitations: the models are based on a limited number of sample projects, and the inputs required for these models can be difficult to assess, often necessitating extensive analysis by trained professionals.

Two big families of algorithmic techniques exist:

1. **Function-based estimations** measure a system in terms of its functions. The most well-known technique in this family is probably the **function point (FP) technique**. A more recent addition includes the object point technique, to mention one
2. **Size-based estimations** measure a system in terms of its physical size, as well as measures in lines of code. The most well-known technique is probably the **constructive cost modeling (COCOMO)** family of models.

Function Points

The FP technique was proposed by Albrecht in the 1970s

FP estimations are based on 19 different characteristics of a system, five of which refer to the functional characteristics of a system and 14 of which refer to nonfunctional aspects.

The five functional measures are

1. **User inputs**: the number of user inputs, or elements in the system which require an input from the user.
2. **User outputs:** the number of user outputs, or elements in the system which produce an output.
3. **User inquiries:** the number of user inputs that generate a software response, such as word count, search result, or software status.
4. **Internal logical files:** the number of files created and used dynamically by the system.
5. **External interfaces:** the number of external files that connect with the software to an external system. For instance, if the software communicates with a device, it is counted as one external interface

The data above are collected and classified according to a three-step complexity scale, which distinguishes among **simple, average, and complex elements**. Thus, for each of the five characteristics described above, we need to produce a threedimensional vector: **number of simple elements, number of average elements, and number of complex elements.**

The technique defines a matrix of weights to take into account the impact that different elements have in determining the complexity of a project. Each pair functional characteristic, complexity has a weight. Thus, for instance, simple user inputs have a weight of 3, average user inputs a weight of 4, and complex user inputs a weight of 5.

The functional size of a system S, called by the method unadjusted function points (UFP), is the weighted sum of the characteristics of the system. It is computed as follows:

$$\text{UFP} = \sum_{i=1}^{5} \begin{bmatrix} k_i^S & k_i^A & k_i^C \end{bmatrix} \cdot \begin{bmatrix} n_i^S \\ n_i^A \\ n_i^C \end{bmatrix}$$

where the index i runs over the five characteristics (user inputs, user outputs, ...); the vector $[n_i^S, n_i^A, n_i^C]$ is the number of simple (^{S}), average (^{A}), and complex (^{C}) elements of type i that we forecast in S. Finally, $[k_i^S, k_i^A, k_i^C]$ is the weight assigned by the method to simple (^{S}), average (^{A}), and complex (^{C}) elements of type i.

UFP provides an estimation of the functional size of the system. Systems with a higher UFP are more complex to develop than systems with a lower UFP. However, UFP does not take into account the complexity deriving from nonfunctional characteristics of a system

Similar to the functional evaluation, managers answer the 14 questions by providing, for each question, a qualitative answer ranging from 0 (irrelevant) to 5 (very influential).

The answers are summed together and added to the magic number 65, thus yielding a value in the range 65–135, where 135 is obtained when the answer to all the questions is 5, since 65 + 5 ∗14 = 135. The value, divided by 100, is called the value adjustment factor, or VAF, and measures the additional (or reduced) effort that is necessary to take care of the implementation of nonfunctional requirements.

Formally,

$$\text{VAF} = \frac{65 + \sum_{i=1}^{14} C_i}{100}$$

VAF is then multiplied by *UFP* to yield the function points (*FP*):

$$\text{FP} = \text{VAF} * \text{UFP}$$

FP measures the complexity of the system to be developed.

Table 3.6 Nonfunctional Characteristics of a System (FP Method)

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated online?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code to be designed reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Once we have the estimation in function points, we can use it to estimate the effort required for the implementation of the system

COCOMO

Constructive cost model (COCOMO) is a family of estimation techniques first introduced by Barry Boehm in the 1980s . IT helps estimate the effort, time, and cost of software development projects.

Basic COCOMO

The Basic COCOMO model provides a rough estimate of the cost, effort, and time required to complete a software project. It uses the size of the project (measured in **thousands of delivered source instructions, KDSI**) to predict the effort.

COCOMO81 is the first COCOMO model.
The basic model can be used when l ittle information is available; the intermediate model can be used when the requirements are defined, and the advanced model can be used when the architecture is sketched.

There are three types of software projects under Basic COCOMO:

- **Organic:** Small, relatively simple projects where teams have good experience with similar projects.
- **Semi-Detached:** Intermediate complexity projects, involving teams with mixed experience levels.
- **Embedded:** Complex projects with stringent requirements and constraints, often hardware-software combined systems.

In more detail, COCOMO81 computes effort and development time as follows:

$$\mathrm{PM} = A_{\mathrm{PM}} \cdot (\mathrm{KSLOC})^{B_{\mathrm{PM}}} \cdot M$$
$$\mathrm{TDEV} = A_{\mathrm{TDEV}} \cdot (\mathrm{PM})^{B_{\mathrm{TDEV}}}$$
$$\mathrm{TEAM} = \mathrm{PM}/\mathrm{TDEV}$$

here PM is the total effort, TDEV is the ideal duration of the project, TEAM is the team size, and KSLOC is the estimated number of thousands of lines code. In all three variants, A and B are constants, while M (which is not required in the basic model) is computed by looking at various project and product characteristics.
The actual values of A and B depend on the overall complexity of the project. COCOMO81, in particular, distinguishes between three types of projects, organic, semidetached, and embedded, according to the project characteristics listed in Table 3.7. For each type of project, the model provides values for APM, ATDEV , BPM, and BTDEV , as shown in Table.

Table COCOMO Base Model

	APM	*BPM*	*ATDEV*	*BTDEV*
Organic	2.40	1.05	2.50	0.38
Semidetached	3.00	1.12	2.50	0.35
Embedded	3.60	1.20	2.50	0.32

Basic COCOMO is useful for quick, rough estimations, but more complex projects require models like Intermediate or Detailed COCOMO.

Example:

If you are estimating the effort for a software project with a size of 10 KSLOC (10,000 lines of delivered source code), and it's a semi-detached project.

COCOMO II (Constructive Cost Model II) is a widely used model for estimating the cost, effort, and schedule for software development projects. It was developed as an update to the original COCOMO model to address the evolving nature of software projects. The main focus of COCOMO II is to handle a broader range of software projects, including object-oriented, agile, and component-based development.

COCOMO II estimation models can be categorized into four main types, as follows:

Application Composition Model

This model is used for estimating effort during prototype development, particularly when reusable components are involved. It operates based on object points and is well-suited for rapid prototyping of systems.

Early Design Model
This model is applied during the system design phase, after the requirements have been gathered. It produces estimates based on function points, which are later converted into lines of source code. The estimates at this stage rely on a simplified algorithm model formula.

Reuse Model
The reuse model estimates the effort required to integrate reusable components or code generated by design or program conversion tools. It handles two types of reused code:

Black-box code: Used when there is no knowledge or modification of the code.
White-box code: Applied when new code is added to the existing one.
The effort for integrating reused code is calculated using specialized formulas.

Post-Architecture Model
This model is used once the system architecture is defined, providing the most detailed and accurate estimates. The post-architecture model calculates the required effort using a comprehensive set of cost drivers and a refined formula to ensure precision.

Scheduling a Plan
Scheduling the plan is composed of the following steps:

- **Identify dependencies among activities**: Determine how activities in the project are related. Some activities can be scheduled freely, while others depend on the completion or start of other tasks, limiting scheduling flexibility.
- **Identify the critical path of the plan:** Pinpoint the most crucial activities whose delay would directly impact the overall project timeline. These critical tasks must be closely monitored to avoid project delays.
- **Allocate resources to tasks and level resources.:** Assign resources to tasks, considering availability and workload limits. Resource leveling adjusts the schedule to account for these constraints, ensuring the plan meets both task dependencies and resource limitations.

Identify Dependencies among Activities
A dependency between two activities A and B defines some kind of constraint in the executability of A and B and imposes a partial ordering on the execution of the activities.

Two types of constraints are relatively common. These are
1. Finish to start (FS). An FS constraint between A and B expresses the fact that B can start only after A is finished. This is probably the most common constraint between activities. For instance, the activity “baking a cake” can start only when all the ingredients have been poured into the baking pot.

2. Start to start (SS). An SS constraint between A and B expresses the fact that B can start only when A starts. For instance, a “monitoring” activity can start only when the activity that is being monitored starts.

Two types of constraints are used and found less often. These are
Finish to finish (FF). An FF constraint between A and B describes a situation in which B can finish only when A finishes.
Start to finish (SF). An SF constraint between A and B describes a situation in which B can finish only when A starts.

Hard and soft constraints. A hard constraint between two activities A and B models a dependency that is in the nature of the work to be performed. A hard constraint cannot be broken without violating the logic of the project.
Soft constraint between two activities A and B can be set as a convenience to simplify project execution or to reduce risks in the project execution. A soft constraint can be broken without violating the logic of the project.

Lag Time: A positive interval. For example, with a lag time of 3 days in a Finish-to-Start (FS) dependency, activity B starts 3 days after activity A finishes.

Lead Time: A negative interval. For example, with a lead time of 3 days in an FS dependency, activity B starts 3 days before activity A finishes.

PERT charts (Program Evaluation and Review Technique) : A PERT chart is a network diagram that allows project managers to create project schedules. They’re used in the Program Evaluation Review Technique (PERT) to represent a project timeline, estimate the duration of tasks, identify task dependencies and find the critical path of a project.

PERT charts show the dependent tasks in your project. These are tasks that can’t start or finish until another task starts or finishes.

Key Components of a PERT Chart

1. **Nodes**: Represent individual tasks or events within a project. Nodes are usually shown as circles or boxes and contain information about each task, such as the task name, expected duration, start and finish dates, and other relevant details.
2. **Links (Arrows)**: Indicate dependencies between tasks. An arrow from one node to another shows that the first task must be completed before the second can begin. This helps to map out the sequence and flow of tasks.
3. **Start and End Nodes**: The chart typically begins with a single start node and ends with a single end node, though complex projects may have multiple starting and ending points.

4. **Critical Path**: The longest path through the network of tasks in terms of time. It represents the sequence of tasks that directly impacts the project's duration. Any delay in tasks on the critical path will delay the entire project.
5. **Time Estimates**: PERT uses three time estimates to account for uncertainty:
    - **Optimistic Time (O)**: The shortest time in which the task can be completed, assuming everything goes as well as possible.
    - **Pessimistic Time (P)**: The longest time the task might take, assuming potential delays or issues.
    - **Most Likely Time (M)**: The time the task is expected to take under normal conditions.
6. **Expected Time (TE)**: A weighted average calculated using the formula:

$$TE = \frac{O + 4M + P}{6}$$

This provides an estimate of the task's duration, factoring in possible variations.

Steps to Create a PERT Chart:

1. **Identify the Tasks**: List all the tasks required to complete the project.
2. **Determine the Sequence**: Identify which tasks depend on others and their order.
3. **Draw the Chart**: Use nodes to represent tasks and arrows to connect them according to their dependencies.
4. **Estimate Time**: Add time estimates for each task.
5.

APPLICATION EXAMPLE
(http://wiki.doing-projects.org/index.php/Scheduling:_Critical_path,_PERT,_Gantt)

Activity	Predecessors
A	-
B	-
C	-
D	A
E	B
F	C
G	C
H	E,F
I	D
J	G,H

Figure 3: Defined tasks and order

Activity	Predecessors	Duration (in days)		
		tO	tP	tM
A	-	5	7	6
B	-	1	5	3
C	-	1	7	4
D	A	1	3	2
E	B	1	9	2
F	C	1	9	5
G	C	2	8	2
H	E,F	4	10	4
I	D	2	8	5
J	G,H	2	8	2

Figure 4: Defined time estimates

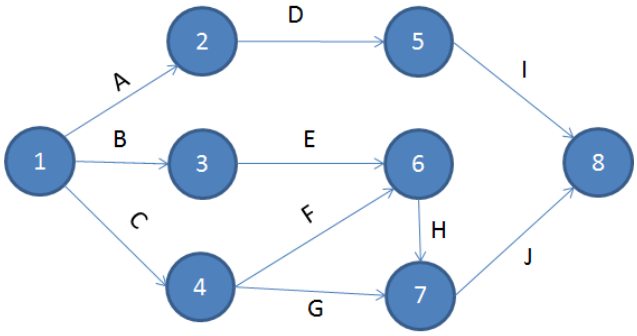
*Determine the estimated expected duration of each activity by using the formula tE = (tO + 4*tM + tP) / 6.*

Lets take activity A as calculation example; tO = 5, tM = 6 and tP = 7.

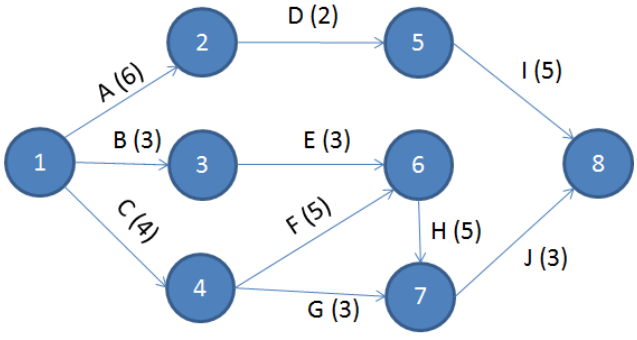
Activity	Predecessors	Duration (in days)			
		tO	tP	tM	tE
A	-	5	7	6	6
B	-	1	5	3	3
C	-	1	7	4	4
D	A	1	3	2	2
E	B	1	9	2	3
F	C	1	9	5	5
G	C	2	8	2	3
H	E,F	4	10	4	5
I	D	2	8	5	5
J	G,H	2	8	2	3

Defined time estimates

Draw and construct the network diagram which connects all the activities/tasks.



Network diagram of the example



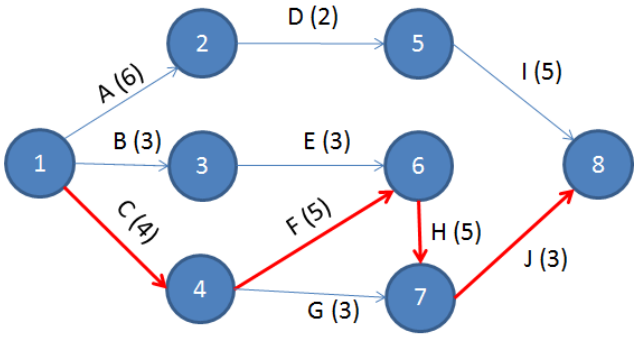
Updated Network diagram of the example including the expected time calculations

Determine the length of the project

We take a look at each path in the network diagram individually. For each path the total length needs to be calculated. Let's define each path and calculate the length of each path:

```
A -> D -> I = 6 + 2 + 5 = 13
B -> E -> H -> J = 3 + 3 + 5 + 3 = 14
C -> F -> H -> J = 4 + 5 + 5 + 3 = 17
C -> G -> J = 4 + 3 + 3 = 10
```

We clearly see that the path C -> F -> H -> J is the

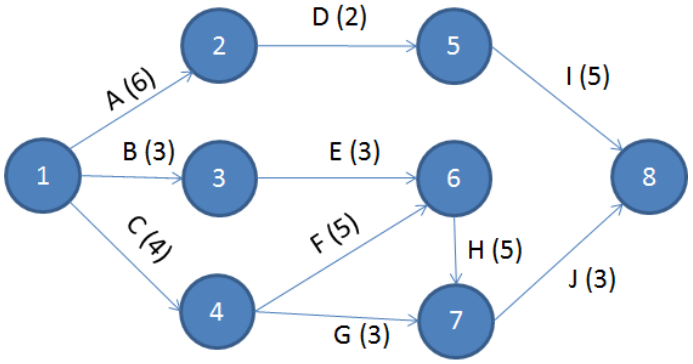
Assign each activity with estimated expected time as calculated in step 3. *Each activity in the network diagram can now be assigned with the calculated tE. The updated network diagram can be seen in Figure*	longest, since the duration is 17 days. We now have the critical path and the network diagram can be updated by marking the path with red as shown in Figure 8.  Network diagram including the critical path

Critical Path algorithm:

1. **Define All Project Activities**
   - List all tasks or activities needed to complete the project.
   - For each task, determine the estimated time required for completion and any dependencies (i.e., which tasks must be completed before others can start).
2. **Construct a Network Diagram**
   - Represent each task as a node or vertex.
   - Connect the nodes with arrows showing dependencies between tasks. This forms a directed acyclic graph where each arrow indicates the flow of work.
3. **Determine Early Start (ES) and Early Finish (EF) Times**
   - Early Start (ES): The earliest time a task can begin based on the completion of its predecessors.
   - Early Finish (EF): The earliest time a task can be completed, calculated as: EF=ES+Duration of the task
   - Begin at the start of the network and move forward through each task, calculating ES and EF for each.
4. **Determine Late Start (LS) and Late Finish (LF) Times**
   - Late Finish (LF): The latest time a task can finish without delaying the project's completion.
   - Late Start (LS): The latest time a task can begin without delaying the project, calculated as: LS=LF−Duration of the task
   - Begin at the end of the network and move backward, calculating LS and LF for each task.
5. **Calculate Slack Time for Each Task**
   - Slack (or float) is the amount of time a task can be delayed without affecting the project's completion time. Slack is calculated as: Slack=LS−ES or Slack=LF−EF
   - A task with zero slack is on the critical path, as any delay in this task will delay the project.
6. **Identify the Critical Path**

- The critical path is the sequence of tasks with zero slack, from the beginning to the end of the network.
- The total duration of the critical path represents the minimum time required to complete the project.

Start from the project finish time, which is the largest EF among all ending activities.if there are multiple ending activities, take the largest EF as the project completion time.

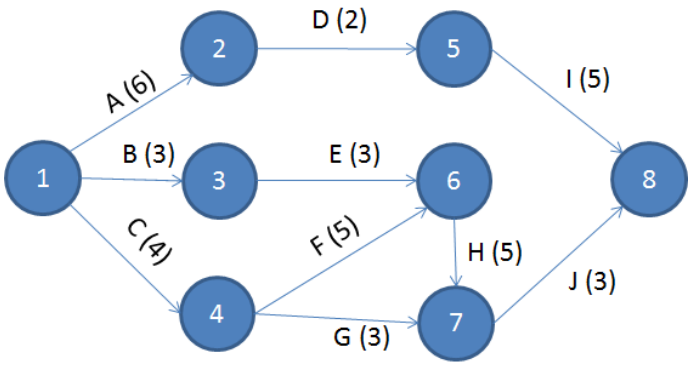


Calculate Early Start (ES) and Early Finish (EF) Times

Starting from the beginning of the network, calculate ES and EF for each activity.

1. Activities with no predecessors start at time 0:
   A: ES=0, EF=ES+6=6
   B: ES=0, EF=ES+3=3
   C: ES=0, EF=ES+4=4
2. Dependent activities:
   D: depends on A. ES=EF of A=6, EF=ES+2=8
   E: depends on B. ES= EF of B =3, EF=ES+3=6
   F: depends on C. ES=EF of C=4, EF=ES+5=9
   G: depends on C. ES=EF of C4, EF=ES+3=7
   H: depends on E AND F. Earliest start is the maximum EF of E and F, so ES=max(6,9)=9, EF=ES+5=14
   J: depends on G and H, Earliest start is the maximum EF of G and H, so ES=max(7,14)=14, EF=ES+3=17
   Activity I (pred = D): ES= EF of D=8
   EF=ES+5=13

Step 3: Calculate Late Finish (LF) and Late Start (LS) Times



Starting from the end of the network (activity J), set LF=EF=17 and move backward to calculate LS and LF for each activity.

Activity J:

- LF=17, LS=LF−3=14

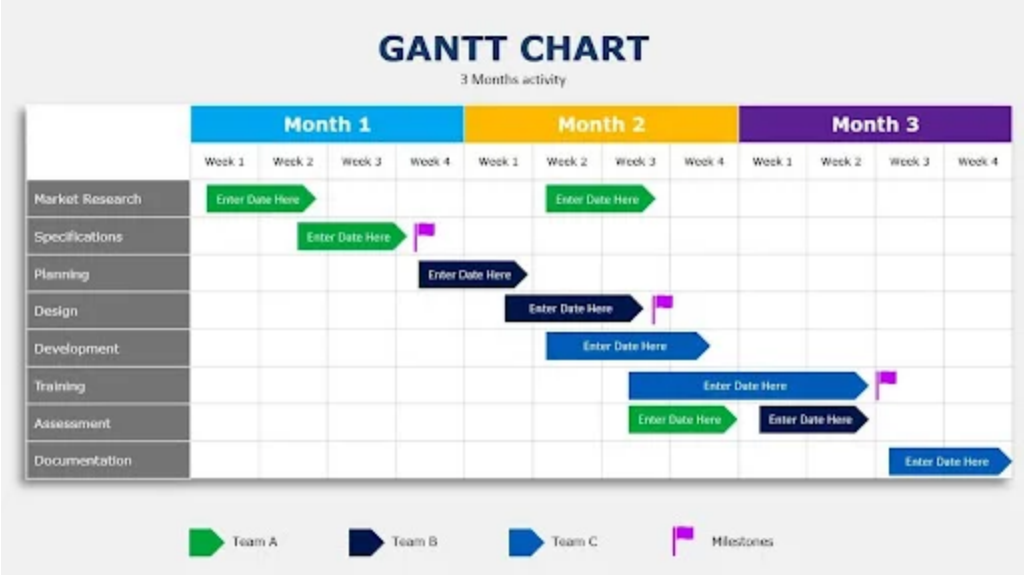
Backward calculations for preceding activities:
G: Depends on J. LF=LS of J=14, LS = LF - 3 = 11
H: Depends on J. LF=LS of J=14, LS=LF−5=9.
I: Depends on no further activities (end of this branch), so ~~LF=EF=13, LS=LF−5=8~~. LF=EF=17, LS=LF-5=12
D: Depends on I. ~~LF=LS of I=8, LS = LF - 2 = 6~~. LF=LS of I=12, LS=LF-2=10
E: Depends on H. LF=LS of H=9, LS=LF−3=6.
F: Depends on H. LF=LS of H=9, LS = LF - 5 = 4.
A: Depends on D. ~~LF=LS of D=6,LS=LF−6=0~~ LF=LS of D=10, LS=LF-6=4
B: Depends on E. LF=LS of E=6, LS=LF−3=3.
C: Depends on F and G. LF=min(LS of F,LS of G)=min(4,11)=4, LS=LF−4=0

Activity	Duration	ES	EF	LF	LS	Slack
A	6	0	6	10	4	4
B	3	0	3	6	3	3
C	4	0	4	4	0	0
D	2	6	8	12	10	4
E	3	3	6	9	6	3
F	5	4	9	9	4	0
G	3	4	7	14	11	7
H	5	9	14	14	9	0
I	5	8	13	17	12	4
J	3	14	17	17	14	0

Critical path is indeed the longest path through the network, which determines the minimum project duration. When multiple paths have activities with zero slack, the longest path among these zero-slack paths is considered the critical path. Any delay along this path will delay the entire project, as there's no slack to absorb any additional time. From the above calculations, **Path 3 (C→F→H→J)** has the longest duration of 17 units. This is the critical path, as it dictates the minimum time required to complete the project.

Gantt Chart?

A **Gantt Chart** is a widely used project management tool that visually represents a project's schedule. It is essentially a bar chart that illustrates a project timeline, showing tasks, their durations, start and end dates, and how they relate to one another.



Key Components of a Gantt Chart

1. **Timeline**: The horizontal axis displays the timeline (days, weeks, months, or even years).
2. **Tasks/Activities**: The vertical axis lists all project tasks or activities that need to be completed.
3. **Bars**: Horizontal bars represent each task. The length of the bar shows the duration of the task, and the position indicates the start and end dates.
4. **Dependencies**: Arrows between bars indicate task dependencies (i.e., which tasks must be completed before others can start).
5. **Milestones**: Special symbols (often diamonds) mark key project events or deliverables.
6. **Progress Tracking**: Shaded portions within the bars can indicate the percentage of work completed.

Optimizing a Plan

In many situations, the project schedule ends up by being too long to respect the constraints set by the stakeholders, by the project goals, or by the environment. This can cause a bit of frustration to the project manager, since, as soon as he or she comes out with a realistic plan, this has to be revised and changed!

Renegotiating Goals and Deadlines

If all the project goals cannot be achieved in the required time frame, renegotiating the project scope and other project constraints can yield a satisfactory solution.

- The simplest renegotiation we can try is on the delivery date.
- Renegotiation is on the project goals: Selecting with the client the most important goals reduces the work we need to do, moving the delivery to an earlier date

Phase the Project: Phasing a project means breaking it into stages so that the most important goals are completed first. In software development, this method works well because it allows for gradual progress. An initial version with basic features is released early, followed by improvements in later phases. This way, users get a working solution quickly, which helps both the team and users understand which features are most important for future development.

Project Crashing: Project Crashing is a project management technique used to shorten the project schedule by accelerating the timeline without significantly increasing costs. The goal is to achieve the project's deadline sooner by adding extra resources, such as manpower, equipment, or budget, to critical tasks. However, it requires careful analysis to balance the trade-offs between cost and time.

- Crash costs indicate the savings obtained by crashing the project.
- Crash time indicates the time used to shorten the project

Fast Tracking
Fast tracking tries to minimize the project duration by breaking the logic of the plan. That is, some of the hard constraints in the plan are removed so that activities that would otherwise be sequential can partially overlap

Critical chain management
- It uses best guesses to estimate each activity (rather cautious estimations) and adds cautious estimations at the ends of the chains, rather than at the end of each activity.
- Instead of adding padding to every task, you would place a **single project buffer** at the end of the critical chain, which can absorb delays that occur during any task in the critical path

Budgeting and Accounting
three of the main aspects of a project: **quality, time, and cost.**

Project costs are the expenses incurred to complete a project.
Costs are divided into two categories:
- Direct Costs: Directly related to the project.
    - Personnel Costs: Costs of the personnel involved, calculated from effort and rates (salary, taxes, benefits, etc.).
    - Materials and Supplies: Costs of materials necessary for the project (e.g., construction materials for building or low material costs for software development).
    - Hardware and Software: Costs for specific hardware and software needed for the project.
    - Travel, Meetings, and Events: Costs related to meetings with customers and stakeholders.
    - Consultants and Subcontracting: Costs related to subcontracted work.

    - Other Costs: Miscellaneous expenses like books, training, or equipment rental.
- Indirect Costs: Necessary to support the project but not directly related to specific tasks.
    - General Overheads: Costs to run the facility and support the production team (e.g., office rent, utilities, administrative staff, and networking).
    - Project Overheads: Costs for project-specific infrastructure, often used in large projects or specific accounting situations. Typically, indirect costs are categorized as general overheads and distributed across all projects within the organization.

Budgeting in Project Management

Budgeting involves estimating, allocating, and controlling the financial resources needed to complete a project within the defined scope, schedule, and quality standards. The budgeting process typically involves the following steps:

1. **Estimation**: The first step is estimating the costs of all resources, tasks, and activities required for the project. This includes labor, materials, equipment, and any other costs directly related to project execution.
    - **Top-down approach**: A rough estimate based on previous projects or industry benchmarks.
    - **Bottom-up approach**: Detailed estimates based on the specific work packages or tasks.
2. **Cost Breakdown Structure (CBS)**: This is the hierarchical decomposition of the project budget into smaller, manageable components. The CBS mirrors the Work Breakdown Structure (WBS) but focuses on costs.
3. **Contingency Planning**: A contingency reserve is typically included in the budget to cover unexpected costs or risks that may arise during the project. This reserve helps manage uncertainties and mitigate financial surprises.
4. **Funding Allocation**: The total budget is distributed across different phases of the project, with funds assigned to specific tasks, milestones, and deliverables.
5. **Cost Baseline**: The approved version of the project budget, which includes all expected costs and the contingency reserve. This becomes the baseline against which actual performance is measured.
6. **Monitoring and Control**: Once the project starts, it's essential to track actual expenses against the budget regularly. Variance analysis is performed to determine if costs are exceeding the budget and to implement corrective actions if necessary.

Table 3.14 Budget Example

	Unit Cost	*Overhead*	*Effort/Units*	*Total*	*Comment*
Personnel					
Resource A	€50	€30	100	€8000	
Resource B	€40	€30	100	€7000	
Total personnel costs				€15,000	
Hardware and software					
Hardware	€300		2	€600	Two tablets for testing the Application
Software	€80		1	€80	Library for graphs
Total hardware and software				€680	
Other costs					
Travel	€1000		5	€5000	
Meetings	€200		3	€600	
Training				€0	
Total other costs				€5000	
Total				€12,280	

Accounting in Project Management

Accounting in project management is the practice of tracking and reporting on the financial transactions of the project. It ensures transparency and accountability in managing the project's finances. Key components include:

1. **Cost Tracking**: Regularly recording all costs incurred during the project's execution, including direct costs (e.g., salaries, materials) and indirect costs (e.g., overheads, administrative costs).
2. **Financial Reporting**: Creating reports that detail the project's financial health, such as:
    - **Cash flow reports**: Show how money is flowing in and out of the project.
    - **Profit and loss statements**: Show if the project is financially on track.
    - **Variance reports**: Compare the budgeted costs to actual expenditures.
3. **Revenue Recognition**: For projects with external clients or deliverables, recognizing revenue when certain milestones or performance criteria are met. This is particularly important for long-term projects with payment structures based on progress.
4. **Audit and Compliance**: Ensuring that the project's financial activities are compliant with relevant accounting standards, regulations, and internal company policies. Regular audits help detect discrepancies, fraud, or financial inefficiencies.
5. **Final Accounting and Closure**: At the conclusion of the project, all financial transactions need to be finalized. This includes reconciling the accounts, paying any remaining obligations, and ensuring that any overrun costs are accounted for. The final report is then presented to stakeholders, documenting how the project's finances were managed.

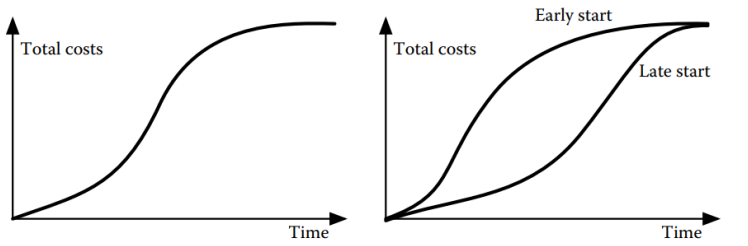


Figure 3.17 Expenditure profiles for a project.

Key Tools and Techniques

- **Earned Value Management (EVM)**: A project performance measurement technique that helps assess cost and schedule performance. Key metrics include:
    - **Planned Value (PV)**: The budgeted cost of work scheduled.
    - **Earned Value (EV)**: The value of work actually performed.
    - **Actual Cost (AC)**: The cost incurred for the work performed.

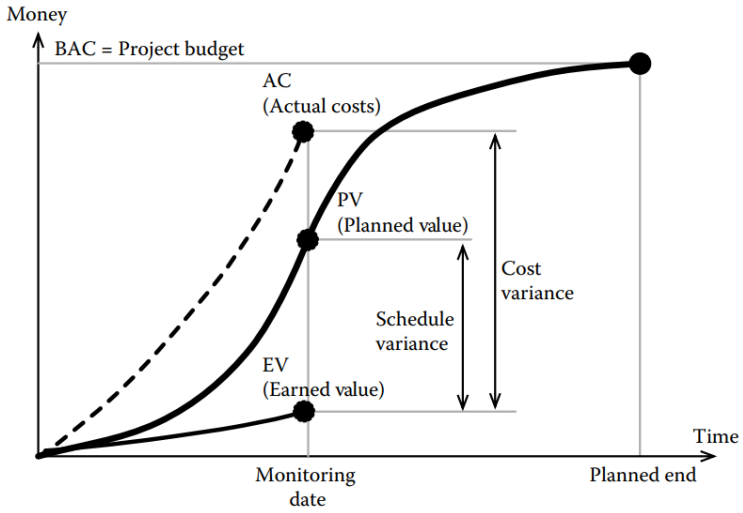


Figure 3.21 Earned value analysis.